

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчет

по лабораторной работе № 10

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний во взвешенном графе»

Выполнили

студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться искать расстояние в взвешенном графе.

Лабораторное задание

Задание 1

Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G . Выведите матрицу на экран

Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки `C++`

*Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.

Определите подмножества периферийных и центральных вершин.

Задание 3*

*Модернизируйте программу так, чтобы получить возможность запуска программы с параметрами командной строки (см. описание ниже). В качестве параметра должны указываться тип графа (взвешенный или нет) и наличие ориентации его ребер (есть ориентация или нет).

Описание метода решения

Метод решения основан на модифицированном алгоритме поиска в ширину (BFS) для взвешенных графов. В отличие от классического BFS, где расстояние до каждой новой вершины увеличивается на единицу, в данной реализации расстояние вычисляется как сумма весов пройденных рёбер. Алгоритм начинает обход из стартовой вершины, помещает её в очередь и устанавливает расстояние до неё равным нулю. На каждом шаге из очереди извлекается текущая вершина, просматриваются все смежные с ней вершины. Для каждой непосещённой смежной вершины расстояние вычисляется как расстояние до текущей вершины плюс вес соединяющего их ребра, после чего эта вершина добавляется в очередь. Таким образом находятся кратчайшие расстояния от начальной вершины до всех остальных в смысле минимальной суммы весов рёбер. На основе этих расстояний вычисляются эксцентриситеты вершин, радиус и диаметр графа, а также определяются центральные и периферийные вершины.

Программа работает аналогично предыдущей. Единственное отличие - адаптация под консоль. Программа поддерживает два режима работы: интерактивный с меню выбора операций и не интерактивный через параметры командной строки, что позволяет гибко настраивать тип графа и выполняемые действия.

Псевдокод

1-ое задание

алг взвешенный_граф_обход_пути

дано

число вершин `vershiny`

граф в виде матрицы смежности `matrica` с весами рёбер

граф в виде списков смежности `spiski` с парами (вершина, вес)

стартовая вершина `start`

надо

сгенерировать случайный взвешенный граф

найти кратчайшие пути от `start` до всех других вершин (методом, похожим на BFS)

выполнить обход в глубину (DFS) от `start`

нач цел

`vershiny`, `start`

`matrica`, `spiski`

`dist`, `poseschena`

// Построение списков смежности из матрицы

процедура `postroitSpiski()`

нач

очистить `spiski`

нц для `i` от 0 до `vershiny-1`

нц для `j` от 0 до `vershiny-1`

если `matrica[i][j] > 0` тогда добавить `{j, matrica[i][j]}` в `spiski[i]`

кц

кц

кон

// Генерация взвешенного графа

процедура generirovat()

нач

инициализировать генератор случайных чисел

обнулить matrica

нц для i от 0 до vershiny-1

нц для j от i+1 до vershiny-1

если случайное число = 1 тогда

ves := случайное число от 1 до 10

matrica[i][j] := ves

matrica[j][i] := ves

конец если

кц

кц

вызвать postroitSpiski()

кон

// Поиск путей от start (метод Дейкстры/BFS на взвешенном графе)

процедура naytiPuti(start)

нач

создать массив dist длины vershiny, заполнить -1

создать очередь q

dist[start] := 0

положить start в q

пока q не пуста

tekushchiy := извлечь из q

нц для i от 0 до vershiny-1

если matrica[tekushchiy][i] > 0 и dist[i] = -1 тогда

```

    dist[i] := dist[tekushchiy] + matrica[tekushchiy][i]
    положить i в q
конец если
кц
кц
вывести "Пути от вершины ", start
нц для i от 0 до vershiny-1
    вывести "До ", i, ": ", dist[i] (или "нет пути")
кц
кон

// Обход в глубину по спискам
процедура obhodVglub(start)
нач
    вывести "Обход в глубину от ", start
    создать массив poseschena длины vershiny, заполнить false
    создать стек s
    положить start в s
    пока s не пуст
        tekushchiy := извлечь из s
        если poseschena[tekushchiy] = false тогда
            вывести tekushchiy
            poseschena[tekushchiy] := true
        конец если
        для каждого соседа в spiski[tekushchiy] (в обратном порядке)
            если poseschena[сосед.first] = false тогда
                положить сосед.first в s
            конец если
        кц
    кц
кц
кон

```

```
// Основная программа

нач
установить локаль
ввести n (количество вершин)
если  $n \leq 0$ ,  $n := 6$ 
создать graf с n вершинами
вызвать graf.pechatMatricy()
вызвать graf.pechatSpiskov()
ввести start (стартовую вершину)
если start в допустимом диапазоне тогда
    вызвать graf.naytiPuti(start)
    вызвать graf.obhodVglub(start)
иначе
    вывести "Неверная вершина!"
конец если
кон
```

2-ое задание

алг анализ_взвешенного_графа

дано

число вершин *vershiny*

тип графа *orientir* (ориентированный или неориентированный)

матрица смежности *matrica* с весами рёбер

надо

сгенерировать случайный взвешенный граф

найти и вывести его радиус, диаметр, центральные и периферийные вершины

нач цел

vershiny, orientir, matrica

ekstsentrizitet, radius, diametr

// Генерация взвешенного графа (ориентированного или неориентированного)

процедура generirovat()

нач

инициализировать генератор случайных чисел

обнулить matrica

нц для i от 0 до vershiny-1

нц для j от 0 до vershiny-1

если i = j тогда пропустить

если orientir тогда

если случайное число > 0 тогда matrica[i][j] := случайный вес (1-10)

иначе если i < j и случайное число > 0 тогда

ves := случайный вес (1-10)

matrica[i][j] := ves, matrica[j][i] := ves

конец если

кц

кц

кон

// Поиск длин путей от вершины start (методом BFS)

функция naytiPuti(start)

нач

создать массив dist длины vershiny, заполнить -1

создать очередь q

dist[start] := 0

положить start в q

пока q не пуста

tekushchiy := извлечь из q


```
нц для i от 0 до vershiny-1
если есть ребро от tekushchiy к i и dist[i] = -1 тогда
dist[i] := dist[tekushchiy] + matrica[tekushchiy][i]
положить i в q
конец если
кц
кц
вернуть dist
кон
```

// Вычисление эксцентриситета вершины

функция getEkstsentrismet(v)

нач

dist := naytiPuti(v)

maxDist := 0

нц для каждого d в dist

если d = -1 тогда вернуть бесконечность // Вершина недостижима

maxDist := max(maxDist, d)

кц

вернуть maxDist

кон

// Анализ графа: радиус, диаметр и соответствующие вершины

процедура analiz()

нач

создать массив ekstsentrismet длины vershiny

radius := бесконечность

diametr := 0

нц для i от 0 до vershiny-1

ekstsentrismet[i] := getEkstsentrismet(i)

если ekstsentrismet[i] ≠ бесконечность тогда

```

radius := min(radius, ekstsentrissety[i])
diametr := max(diametr, ekstsentrissety[i])
конец если
кц
вывести "Радиус: ", radius
вывести "Диаметр: ", diametr

вывести "Центральные вершины: "
нц для i от 0 до vershiny-1
если ekstsentrissety[i] = radius тогда вывести i
кц

вывести "Периферийные вершины: "
нц для i от 0 до vershiny-1
если ekstsentrissety[i] = diametr тогда вывести i
кц
кон

```

// Основная программа

нач

установить локаль

ввести n (количество вершин)

если $n \leq 0$ тогда $n := 6$

ввести tip (1 - неориентированный, 2 - ориентированный)

orientir := (tip = 2)

создать graf с n вершинами и типом orientir

вызвать graf.pechatMatricy()

вызвать graf.analiz()

кон

3-ие задание

дано

число вершин `vershiny`

тип графа `orientir` (ориентированный или нет)

матрица смежности `matrica` с весами

аргументы командной строки `argc`, `argv`

надо

сгенерировать граф на основе входных данных

выполнить анализ графа (радиус, диаметр, центры, периферия)

найти пути от стартовой вершины

предоставить интерактивное меню для управления

нач цел

`vershiny`, `start`, `vybor`, `matrica`

`orientir`, `delatAnaliz`, `interaktiv`

// Обработка аргументов командной строки

процедура `obrabotatKomandy(argc, argv, &vershiny, &orientir, &delatAnaliz, &start, &interaktiv)`

нач

`vershiny := 6`, `orientir := ложь`, `delatAnaliz := ложь`, `start := -1`, `interaktiv := (argc = 1)`

нц для `i` от 1 до `argc-1`

если `argv[i] = "-v"` тогда `vershiny := число из argv[i+1]`

иначе если `argv[i] = "-d"` тогда `orientir := истина`

иначе если `argv[i] = "-u"` тогда `orientir := ложь`

иначе если `argv[i] = "-a"` тогда `delatAnaliz := истина`

иначе если `argv[i] = "-s"` тогда `start := число из argv[i+1]`

кц

если `delatAnaliz` или `start ≠ -1` тогда `interaktiv := ложь`

кон

```

// Генерация взвешенного графа
процедура generirovat()
нач
инициализировать генератор случайных чисел
обнулить matrica
нц для i от 0 до vershiny-1
нц для j от 0 до vershiny-1
если i = j тогда пропустить
если orientir и случайное > 0 тогда matrica[i][j] := случайный вес
иначе если не orientir и i < j и случайное > 0 тогда
ves := случайный вес
matrica[i][j] := ves, matrica[j][i] := ves
конец если
кц
кц
кон

// Поиск длин путей от вершины start
функция naytiPuti(start)
нач
создать массив dist длины vershiny, заполнить -1
создать очередь q
dist[start] := 0
положить start в q
пока q не пуста
tek := извлечь из q
нц для i от 0 до vershiny-1
если matrica[tek][i] > 0 и dist[i] = -1 тогда
dist[i] := dist[tek] + matrica[tek][i]
положить i в q
конец если

```

кц

кц

вернуть dist

кон

// Вывод путей

процедура vyvestiPuti(start)

нач

dist := naytiPuti(start)

вывести "Пути от вершины ", start

нц для i от 0 до vershiny-1

вывести "До ", i, ": ", (dist[i] = -1 ? "нет пути" : dist[i])

кц

кон

// Вычисление эксцентриситета

функция getEksc(v)

нач

dist := naytiPuti(v)

maxDist := 0

нц для каждого d в dist

если d = -1 тогда вернуть бесконечность

maxDist := max(maxDist, d)

кц

вернуть maxDist

кон

// Анализ графа

процедура analiz()

нач

ekscVseh := массив длины vershiny

radius := бесконечность, diametr := 0

нц для i от 0 до vershiny-1

ekscVseh[i] := getEksc(i)

если ekscVseh[i] $\neq$ бесконечность тогда

radius := min(radius, ekscVseh[i])

diametr := max(diametr, ekscVseh[i])

конец если

кц

вывести эксцентриситеты, радиус, диаметр

вывести центральные вершины (где эксцентриситет = радиус)

вывести периферийные вершины (где эксцентриситет = диаметр)

кон

// Основная программа

нач

установить локаль

вызвать obrabotatKomandy

если vershiny $\leq$ 0 тогда vershiny := 6

создать graf с vershiny и orientir

если не interaktiv тогда

вызвать graf.pechatMatricy()

если delatAnaliz тогда вызвать graf.analiz()

если start $\neq$ -1 тогда вызвать graf.vyvestiPuti(start)

завершить программу

конец если

нц

вывести меню (1-неориентир., 2-ориентир., 3-анализ, 4-изменить размер, 0-
выход)

ввести vybor

выбор vybor из

случай 1: graf.setOrientir(ложь), pechatMatricy(),

запросить start, vyvestiPuti(start)

случай 2: graf.setOrientir(истина), pechatMatricy(),

запросить start, vyvestiPuti(start)

случай 3: pechatMatricy(), analiz()

случай 4:

запросить novyeVershiny, graf.setVershiny(novyeVershiny), pechatMatricy()

случай 0: ВЫЙТИ ИЗ ЦИКЛА

КОНЕЦ ВЫБОРА

кц пока vybor = 0

КОН

Листинг

1-ое задание

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <random>
#include <iomanip>

using namespace std;

class Graf {
private:
    vector<vector<int>> matrica;
    vector<vector<pair<int, int>>> spiski;
    int vershiny;

    void postroitSpiski() {
        spiski.assign(vershiny, vector<pair<int, int>>());
        for (int i = 0; i < vershiny; ++i) {
            for (int j = 0; j < vershiny; ++j) {
                if (matrica[i][j] > 0) {
                    spiski[i].push_back({ j, matrica[i][j] });
                }
            }
        }
    }

public:
    Graf(int n) : vershiny(n) {
        generirovat();
        postroitSpiski();
    }
};
```



```
}
```

```
void generirovat() {  
    random_device rd;  
    mt19937 gen(rd());  
    uniform_int_distribution<> distVes(1, 10);  
    uniform_int_distribution<> estRebro(0, 1);  
  
    matrica.assign(vershiny, vector<int>(vershiny, 0));  
  
    for (int i = 0; i < vershiny; i++) {  
        for (int j = i + 1; j < vershiny; j++) {  
            if (estRebro(gen)) {  
                int ves = distVes(gen);  
                matrica[i][j] = ves;  
                matrica[j][i] = ves;  
            }  
        }  
    }  
}
```

```
void pechatMatricy() {  
    cout << "\nMatrica grafa:\n    ";  
    for (int i = 0; i < vershiny; i++) cout << setw(4) << i;  
    cout << "\n";  
    for (int i = 0; i < vershiny; i++) {  
        cout << setw(4) << i << " ";  
        for (int j = 0; j < vershiny; j++) cout << setw(4)  
<< matrica[i][j];  
        cout << "\n";  
    }  
}
```

```

void pechatSpiskov() {
    cout << "\nSpiski smezhnosti:\n";
    for (int i = 0; i < vershiny; i++) {
        cout << i << ": ";
        for (const auto& sosed : spiski[i]) {
            cout << sosed.first << "(" << sosed.second << "
";

        }
        cout << "\n";
    }
}

void naytiPuti(int start) {
    vector<int> dist(vershiny, -1);
    queue<int> q;

    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int tekushchiy = q.front();
        q.pop();

        for (int i = 0; i < vershiny; i++) {
            if (matrica[tekushchiy][i] > 0 && dist[i] == -1)
            {
                dist[i] = dist[tekushchiy] +
matrica[tekushchiy][i];
                q.push(i);
            }
        }
    }
}

```

```
        cout << "\nPuti ot vershiny " << start << " (BFS-  
metod):\n";
```

```
        for (int i = 0; i < vershiny; i++) {  
            cout << "Do " << i << ": ";  
            if (dist[i] == -1) cout << "net puti";  
            else cout << dist[i];  
            cout << "\n";  
        }  
    }
```

```
void obhodVglub(int start) {  
    cout << "\nObhod v glubinu po spiskam (DFS) ot " <<  
start << ":\n";  
    vector<bool> poseschena(vershiny, false);  
    stack<int> s;  
    s.push(start);  
  
    while (!s.empty()) {  
        int tekushchiy = s.top();  
        s.pop();  
  
        if (!poseschena[tekushchiy]) {  
            cout << tekushchiy << " ";  
            poseschena[tekushchiy] = true;  
        }  
  
        for (auto it = spiski[tekushchiy].rbegin(); it !=  
spiski[tekushchiy].rend(); ++it) {  
            if (!poseschena[it->first]) {  
                s.push(it->first);  
            }  
        }  
    }
```

```

        }

        cout << endl;
    }
};

int main() {
    setlocale(LC_ALL, "Russian");
    int n;
    cout << "Vvedite kolichestvo vershin: ";
    cin >> n;
    if (n <= 0) n = 6;

    Graf graf(n);
    graf.pechatMatricy();
    graf.pechatSpiskov();

    int start;
    cout << "\nVvedite startovuyu vershinu: ";
    cin >> start;

    if (start >= 0 && start < n) {
        graf.naytiPuti(start);
        graf.obhodVglub(start);
    }
    else {
        cout << "Nevernaya vershina!\n";
    }
    return 0;
}

```

2-ое задание

```
#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <limits>
#include <algorithm>
#include <string>

using namespace std;

class Graf {
private:
    vector<vector<int>> matrica;
    int vershiny;
    bool orientir;

    vector<int> naytiPuti(int start) {
        vector<int> dist(vershiny, -1);
        queue<int> q;
        dist[start] = 0;
        q.push(start);

        while (!q.empty()) {
            int tekushchiy = q.front();
            q.pop();
            for (int i = 0; i < vershiny; i++) {
                if (matrica[tekushchiy][i] > 0 && dist[i] == -1)
                {
                    dist[i] = dist[tekushchiy] +
matrica[tekushchiy][i];
                    q.push(i);
                }
            }
        }
    }
};
```

```

        }
    }
}

return dist;
}

public:
    Graf(int n, bool isOriented) : vershiny(n),
orientir(isOriented) {
        generirovat();
    }

    void generirovat() {
        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> distVes(1, 10);
        uniform_int_distribution<> estRebro(0, 2);

        matrica.assign(vershiny, vector<int>(vershiny, 0));

        for (int i = 0; i < vershiny; i++) {
            for (int j = 0; j < vershiny; j++) {
                if (i == j) continue;
                if (orientir) {
                    if (estRebro(gen) > 0) matrica[i][j] =
distVes(gen);
                }
                else if (i < j && estRebro(gen) > 0) {
                    int ves = distVes(gen);
                    matrica[i][j] = matrica[j][i] = ves;
                }
            }
        }
    }
}

```

```
}
```

```
void pechatMatricy() {  
    cout << "\nMatrica " << (orientir ? "orientirovannogo" :  
"neorientirovannogo") << " grafa:\n    ";  
    for (int i = 0; i < vershiny; i++) cout << setw(4) << i;  
    cout << "\n";  
    for (int i = 0; i < vershiny; i++) {  
        cout << setw(4) << i << " ";  
        for (int j = 0; j < vershiny; j++) cout << setw(4)  
<< matrica[i][j];  
        cout << "\n";  
    }  
}
```

```
int getEkstsentrisset(int v) {  
    vector<int> dist = naytiPuti(v);  
    int maxDist = 0;  
    for (int d : dist) {  
        if (d == -1) return numeric_limits<int>::max();  
        maxDist = max(maxDist, d);  
    }  
    return maxDist;  
}
```

```
void analiz() {  
    vector<int> ekstsentrissety(vershiny);  
    int radius = numeric_limits<int>::max();  
    int diametr = 0;  
  
    for (int i = 0; i < vershiny; i++) {  
        ekstsentrissety[i] = getEkstsentrisset(i);
```

```

        if (ekstsentrality[i] !=
numeric_limits<int>::max()) {
            radius = min(radius, ekstsentrality[i]);
            diametr = max(diametr, ekstsentrality[i]);
        }
    }

    cout << "\nAnaliz grafa\n";

    cout << "Radius: " << (radius ==
numeric_limits<int>::max() ? "ne opredelen" : to_string(radius))
<< endl;

    cout << "Diametr: " << (diametr == 0 ? "ne opredelen" :
to_string(diametr)) << endl;

    cout << "Tsentralnye vershiny: ";
    for (int i = 0; i < vershiny; i++) if
(ekstsentrality[i] == radius) cout << i << " ";
    cout << endl;

    cout << "Periferiynye vershiny: ";
    for (int i = 0; i < vershiny; i++) if
(ekstsentrality[i] == diametr) cout << i << " ";
    cout << endl;
}

};

int main() {
    setlocale(LC_ALL, "Russian");
    int n, tip;
    cout << "Vvedite kolichestvo vershin: ";
    cin >> n;
    if (n <= 0) n = 6;

    cout << "\nVyberite tip grafa (1 - neorient., 2 - orient.):
";

```



```

        cin >> tip;

        Graf graf(n, tip == 2);
        graf.pechatMatricy();
        graf.analiz();

        return 0;
    }

```

3-ие задание

```

#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <limits>
#include <algorithm>
#include <string>
#include <cstring>

using namespace std;

class Graf {
private:
    vector<vector<int>> matrica;
    int vershiny;
    bool orientir;

    vector<int> naytiPuti(int start) {
        if (start < 0 || start >= vershiny) {
            return vector<int>(vershiny, -1);
        }
        vector<int> dist(vershiny, -1);

```

```

queue<int> q;
dist[start] = 0;
q.push(start);

while (!q.empty()) {
    int tek = q.front();
    q.pop();
    for (int i = 0; i < vershiny; i++) {
        if (matrica[tek][i] > 0 && dist[i] == -1) {
            dist[i] = dist[tek] + matrica[tek][i];
            q.push(i);
        }
    }
}
return dist;
}

```

public:

```

    Graf(int n, bool isOriented = false) : vershiny(n),
orientir(isOriented) {
        generirovat();
    }

```

```

void generirovat() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> distVes(1, 10);
    uniform_int_distribution<> distRebro(0, 2);

    matrica.assign(vershiny, vector<int>(vershiny, 0));

    for (int i = 0; i < vershiny; i++) {

```

```

        for (int j = 0; j < vershiny; j++) {
            if (i == j) continue;
            if (orientir) {
                if (distRebro(gen) > 0) matrica[i][j] =
distVes(gen);
            }
            else if (i < j && distRebro(gen) > 0) {
                int ves = distVes(gen);
                matrica[i][j] = matrica[j][i] = ves;
            }
        }
    }
}

```

```

void pechatMatricy() {
    cout << "\nMatrica " << (orientir ? "orientirovannogo" :
"neorientirovannogo") << " grafa:\n";
    cout << "    ";
    for (int i = 0; i < vershiny; i++) cout << setw(4) << i;
    cout << "\n";
    for (int i = 0; i < vershiny; i++) {
        cout << setw(3) << i;
        for (int j = 0; j < vershiny; j++) cout << setw(4)
<< matrica[i][j];
        cout << "\n";
    }
}

```

```

void vyvestiPuti(int start) {
    vector<int> dist = naytiPuti(start);
    cout << "Puti ot vershiny " << start << ":\n";
    for (int i = 0; i < vershiny; i++) {
        cout << "Do " << i << ": ";
    }
}

```

```

        if (dist[i] == -1) cout << "net puti";
        else cout << dist[i];
        cout << "\n";
    }
}

int getEksc(int v) {
    if (v < 0 || v >= vershiny) return -1;
    vector<int> dist = naytiPuti(v);
    int maxDist = 0;
    for (int d : dist) {
        if (d == -1) return numeric_limits<int>::max();
        maxDist = max(maxDist, d);
    }
    return maxDist;
}

void analiz() {
    vector<int> ekscVseh(vershiny);
    int radius = numeric_limits<int>::max();
    int diametr = 0;

    for (int i = 0; i < vershiny; i++) {
        ekscVseh[i] = getEksc(i);
        if (ekscVseh[i] != numeric_limits<int>::max()) {
            radius = min(radius, ekscVseh[i]);
            diametr = max(diametr, ekscVseh[i]);
        }
    }

    vector<int> tscopy, periferiya;
    for (int i = 0; i < vershiny; i++) {

```

```

        if (ekscVseh[i] == radius) tentry.push_back(i);
        if (ekscVseh[i] == diametr) periferiya.push_back(i);
    }

    cout << "\nEkstsentrishiteti:\n";
    for (int i = 0; i < vershiny; i++) {
        cout << "Vershina " << i << ": " << (ekscVseh[i] ==
numeric_limits<int>::max() ? "inf" : to_string(ekscVseh[i])) <<
"\n";
    }

    cout << "\nRadius grafa: " << (radius ==
numeric_limits<int>::max() ? "ne opredelen" : to_string(radius))
<< endl;

    cout << "Diametr grafa: " << (diametr == 0 ? "ne
opredelen" : to_string(diametr)) << endl;

    cout << "Tsentralnye vershiny: ";
    for (int v : tentry) cout << v << " ";
    cout << endl;

    cout << "Periferiynye vershiny: ";
    for (int v : periferiya) cout << v << " ";
    cout << endl;
}

int getVershiny() const { return vershiny; }
bool isOrientir() const { return orientir; }
void setVershiny(int n) { vershiny = n; generirovat(); }
void setOrientir(bool isOriented) { orientir = isOriented;
generirovat(); }
};

void obrabotatKomandy(int argc, char* argv[], int& vershiny,
bool& orientir, bool& delatAnaliz, int& start, bool& interaktiv)
{
    vershiny = 6;
    orientir = false;

```

```

    delatAnaliz = false;

    start = -1;

    interaktiv = (argc == 1);

    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-v") == 0 && i + 1 < argc) vershiny
= atoi(argv[++i]);
        else if (strcmp(argv[i], "-d") == 0) orientir = true;
        else if (strcmp(argv[i], "-u") == 0) orientir = false;
        else if (strcmp(argv[i], "-a") == 0) delatAnaliz = true;
        else if (strcmp(argv[i], "-s") == 0 && i + 1 < argc)
start = atoi(argv[++i]);
    }

    if (delatAnaliz || start != -1) interaktiv = false;
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");

    int vershiny, start, novyeVershiny, vybor;

    bool orientir, delatAnaliz, interaktiv;

    obrabotatKomandy(argc, argv, vershiny, orientir,
delatAnaliz, start, interaktiv);

    if (vershiny <= 0) vershiny = 6;

    Graf graf(vershiny, orientir);

    if (!interaktiv) {
        graf.pechatMatricy();

        if (delatAnaliz) graf.analiz();

        if (start != -1) {

```

```

        if (start >= 0 && start < graf.getVershiny())
graf.vyvestiPuti(start);

        else cout << "Nevernaya startovaya vershina: " <<
start << endl;

    }

    return 0;

}

do {

    cout << "\n1. Neorientirovanny graf";
    cout << "\n2. Orientirovanny graf";
    cout << "\n3. Analiz grafa";
    cout << "\n4. Izmenit' kolichestvo vershin";
    cout << "\n0. Vyhod";
    cout << "\nVyberite operatsiyu: ";
    cin >> vybor;
    switch (vybor) {
    case 1:

        if (graf.isOrientir()) graf.setOrientir(false);
        graf.pechatMatricy();
        cout << "Vvedite startovuyu vershinu: ";
        cin >> start;

        if (start >= 0 && start < graf.getVershiny())
graf.vyvestiPuti(start);

        else cout << "Nevernaya vershina!\n";
        break;
    case 2:

        if (!graf.isOrientir()) graf.setOrientir(true);
        graf.pechatMatricy();
        cout << "Vvedite startovuyu vershinu: ";
        cin >> start;

        if (start >= 0 && start < graf.getVershiny())
graf.vyvestiPuti(start);

```

```

        else cout << "Nevernaya vershina!\n";
        break;
    case 3:
        graf.pechatMatricy();
        graf.analiz();
        break;
    case 4:
        cout << "Vvedite novoe kolichestvo vershin: ";
        cin >> novyeVershiny;
        if (novyeVershiny > 0) {
            graf.setVershiny(novyeVershiny);
            graf.pechatMatricy();
        }
        else cout << "Nevernoe kolichestvo!\n";
        break;
    }
} while (vybor != 0);
return 0;
}

```


Результат работы программы

1-ое:

```
Консоль отладки Microsoft V  X  +  -  кадров/с N/A N/A | ГП 1% 36°C 525 МГц 0.815 В 21 Вт 0 об/мин 140
Vvedite kolichество verшин: 5

Matrica grafa:
  0  1  2  3  4
0  0  0  0  0  1
1  0  0  0  0  0
2  0  0  0  0  7
3  0  0  0  0  4
4  1  0  7  4  0

Spiski smezhnosti:
0: 4(1)
1:
2: 4(7)
3: 4(4)
4: 0(1) 2(7) 3(4)

Vvedite startovuyu vershinu: 1

Puti ot vershiny 1 (BFS-metod):
Do 0: net puti
Do 1: 0
Do 2: net puti
Do 3: net puti
Do 4: net puti

Obhod v glubinu po spiskam (DFS) ot 1:
1
```

2-ое:

```
Консоль отладки Microsoft Vi X + v
Vvedite kolichество verшин: 5
Vyberite tip grafa (1 - neorient., 2 - orient.): 1
Matrica neorientirovannogo grafa:
  0  1  2  3  4
0  0  3 10  9  3
1  3  0  4  1 10
2 10  4  0  6  2
3  9  1  6  0  1
4  3 10  2  1  0

Analiz grafa
Radius: 9
Diametr: 10
Tsentralnye verшины: 3
Periferiynye verшины: 0 1 2 4
```

```
Консоль отладки Microsoft Vi X + v
Vvedite kolichество verшин: 5
Vyberite tip grafa (1 - neorient., 2 - orient.): 2
Matrica orientirovannogo grafa:
  0  1  2  3  4
0  0  8  0  9  8
1  6  0  3  2  4
2  2  0  0  9 10
3  8  0  0  0  0
4  7  7  1  3  0

Analiz grafa
Radius: 6
Diametr: 19
Tsentralnye verшины: 1
Periferiynye verшины: 3
```

3-ие:

1. Neorientirovanny graf
 2. Orientirovanny graf
 3. Analiz grafa
 4. Izmenit' kolichestvo vershin
 0. Vyhod
- Vyberite operatsiyu: 1

Matrica neorientirovannogo grafa:

	0	1	2	3	4	5
0	0	8	9	0	10	7
1	8	0	2	0	8	9
2	9	2	0	6	9	6
3	0	0	6	0	2	0
4	10	8	9	2	0	3
5	7	9	6	0	3	0

Vvedite startovuyu vershinu: 2

Puti ot vershiny 2:

Do 0: 9

Do 1: 2

Do 2: 0

Do 3: 6

Do 4: 9

Do 5: 6

1. Neorientirovanny graf
 2. Orientirovanny graf
 3. Analiz grafa
 4. Izmenit' kolichestvo vershin
 0. Vyhod
- Vyberite operatsiyu: 3

Matrica neorientirovannogo grafa:

	0	1	2	3	4	5
0	0	8	9	0	10	7
1	8	0	2	0	8	9
2	9	2	0	6	9	6
3	0	0	6	0	2	0
4	10	8	9	2	0	3
5	7	9	6	0	3	0

Ekstsentrizitet:

Vershina 0: 15

Vershina 1: 9

Vershina 2: 9

Vershina 3: 15

Vershina 4: 10

Vershina 5: 12

Radius grafa: 9

Diametr grafa: 15

Вывод

В ходе лабораторной работы был реализован алгоритм поиска расстояний во взвешенных графах на основе модифицированного обхода в ширину. Программа позволяет генерировать случайные ориентированные и неориентированные графы, находить кратчайшие пути и анализировать основные характеристики графа - радиус, диаметр, эксцентриситеты вершин. Реализация поддерживает два режима работы: интерактивный с меню выбора операций и пакетный через параметры командной строки. Тестирование подтвердило корректность работы алгоритма для графов различной структуры и размера.